WEB TECHNOLOGIES & FULL-STACK SYSTEMS LABORATORY

# CreditRiskSystem

An Institutional-Grade Credit Risk Assessment & Quantitative Investment Evaluation Platform featuring Real-Time Scoring, Wealth Analytics, and Secured Management.

HTML5 (Templates) | CSS3 / SCSS (Glassmorphism) | JavaScript (ES6+) | AJAX (HttpClient / RxJS) | TypeScript (Signals)

XML / JSON APIs | SQL Server / EF Core | ASP.NET Core 8 Web API | Angular 18+ Standalone

**PROJECT DOMAIN**
FinTech • Credit Scoring • Quantitative Finance

**COURSE / EXAMINATION**
Web Technology Laboratory Examination

**FRONTEND ARCHITECTURE**
Angular 18+ SPA (HTML5, SCSS, TypeScript, AJAX)

**BACKEND & DATABASE**
ASP.NET Core 8 REST API • MS SQL Server (EF Core)

**PRIMARY EVALUATION FOCUS**
HTML, CSS/SCSS, JS, AJAX, XML/JSON, TS & Database

**DOCUMENTATION DATE**
Academic Year 2024–2026 • Final Submission

## 1 Executive Summary & Real-World Utility

**CreditRiskSystem** is an enterprise-class financial technology platform engineered to bridge the critical gap between complex institutional credit underwriting and consumer loan decision-making. In traditional banking workflows, credit risk analysis is slow, opaque, heavily manual, and disconnected from dynamic macroeconomic market conditions. Furthermore, retail borrowers and small businesses rarely possess the tools to quantify the true opportunity cost of taking high-interest debt versus investing equivalent capital into wealth-generating mutual funds and systematic investment plans (SIPs).

### ⚠️ What Problem Does It Solve?

- **Opaque Bank Decisions:** Borrowers often face loan rejection without actionable insights into why they failed or what financial metric triggered the adverse decision.
- **Slow & Manual Underwriting:** Manual appraisal of debt-to-income ratios and balance sheet leverage requires days of back-and-forth documentation.
- **Lack of Opportunity Cost Awareness:** Borrowers fail to calculate whether borrowing ₹5,00,000 at 9.5% costs them more than what an equivalent mutual fund SIP would compound over 10 years.
- **Static Risk Metrics:** Conventional credit scores ignore real-time macroeconomic shifts (Repo rate fluctuations, inflation spikes, gold volatility).

### 💡 How Does It Help the User & Bank?

- **Instant Multi-Factor Risk Scoring:** Generates a unified 0–100 Composite Score in under 200ms using 5 institutional mathematical models.
- **Transparent Likelihood Breakdown:** Tells the applicant whether banks will *Approve*, *Conditionally Review*, or *Reject*, explaining the debt-to-income and volatility factors.
- **Wealth Growth & Comparative Engine:** Directly pits loan repayments against systematic investment returns (SIP vs. Loan) using Sharpe-adjusted metrics.
- **Financial Education:** A dynamic interactive encyclopedia (`/learn`) explaining quantitative finance models with formulas and real-world examples.

> **Core Mission of CreditRiskSystem**
>
> To democratize quantitative financial risk analysis by delivering a modern, responsive, glassmorphic Single Page Application (SPA) that empowers both loan applicants and credit underwriters to make mathematically proven, data-backed lending and investment decisions.

## 2 Platform Modules & Core Capabilities

The platform consists of five tightly integrated functional modules accessible via a responsive Single Page Application navigation bar:

| Module & Route | Target User | Key Features & Functionality | Mathematical Models Used |
|---|---|---|---|
| **Loan Profiling** `/apply` | Loan Applicants & Underwriters | Multi-step dynamic form collecting income, existing debt, credit score, employment category, loan amount, and tenure. Provides real-time EMI calculation and instant bank likelihood decisioning. | Sharpe Ratio, CAPM, Kelly Criterion, Value at Risk (VaR), Merton Model |
| **Investment Hub** `/invest` | Retail & Wealth Investors | 3-in-1 interactive calculator suite: (1) SIP future corpus calculator with interest gain breakdown, (2) Real CAGR calculator, and (3) Monte Carlo NAV trajectory simulation over 500 stochastic paths. | SIP Annuity Formula, CAGR Compound Formula, Geometric Brownian Motion (GBM) |
| **Loan vs Fund** `/compare` | Decision-Making Borrowers | Pits capital allocation into loan repayment against equity fund accumulation. Compares net wealth generated, Sharpe efficiency, and provides an algorithmic recommendation on which option wins. | Sharpe Ratio, Black-Scholes Volatility, Kelly Bet Sizing |
| **Knowledge Hub** `/learn` | Students & Financial Analysts | Categorized, searchable, and expandable repository of quantitative models with mathematical formulas, summaries, and industry use cases seeded dynamically from the database. | All 12 Platform Quantitative Models |
| **Admin Center** `/admin` | System Administrators & Risk Heads | Secured JWT authentication portal with live session countdown, platform analytics (approval rates, avg score, volume), audit logs of all runs, and live macroeconomic control (Repo rate, Inflation, Gold). | JWT Cryptographic Signature, System Aggregations, DB Audit Logging |

## 3 Complete End-to-End System Workflow

The system follows a reactive, decoupled client-server architecture. Below is the step-by-step lifecycle of a user performing a comprehensive credit risk assessment:

**1** **Client-Side User Input & Form Progression (Angular HTML5 / TypeScript)**

The user navigates to `/apply`. Step 1 collects applicant financial health (Annual Income, Existing Debt, Credit Score, Employment Type). In Step 2, the user inputs requested loan parameters (Loan Amount, Term, Rate). As values are entered, client-side TypeScript computed getters dynamically calculate Debt-to-Income (DTI) ratio, estimated default probability, and monthly EMI in real-time.

**2** **Asynchronous Request Dispatch (AJAX / HttpClient)**

Upon clicking *"Analyse Profile"*, the component sets `loading.set(true)`, which instantly renders a visual loading skeleton. The Angular `FinanceService` sends an asynchronous HTTP POST request via `HttpClient` to `/api/finance/loan-risk-profile` with a JSON payload containing normalized parameters (risk-free rate, market return, employment beta mapping, applicant assets, and asset volatility).

**3** **REST API Controller Ingestion & Model Binding (ASP.NET Core 8)**

The request arrives at `FinanceController.LoanRiskProfile()`. ASP.NET Core deserializes the incoming JSON payload into a strongly-typed `LoanRiskProfileRequest` DTO. The controller invokes the high-performance C# `FinanceEngineService`.

**4** **Multi-Model Quantitative Finance Computation (Backend Finance Engine)**

The Finance Engine executes 5 institutional algorithms in sequence:

- **Sharpe Calculator:** Evaluates loan return efficiency relative to risk-free rate and volatility.
- **CAPM Calculator:** Computes the hurdle rate of return based on employment stability beta (0.6 for Salaried, 1.3 for Business).
- **Kelly Calculator:** Determines optimal lending fraction based on default probability and payoff ratio.
- **VaR Calculator:** Calculates the Value-at-Risk monetary loss exposure at 95% confidence over 30 days.
- **Merton Calculator:** Evaluates structural distance-to-default treating applicant assets and liabilities under Black-Scholes option theory.

A weighted algorithm normalizes all signals into a 0–100 **Composite Risk Score** and determines the institutional decision: **Approve (Score ≥ 70)**, **Review (Score 40–69)**, or **Reject (Score < 40)**.

**5** **Database Audit Logging (Entity Framework Core / SQL Server)**

Before returning the response, the controller instantiates an `AnalysisLog` entity, serializing the full input parameters and output results into JSON strings along with the composite score, decision, and timestamp, and persists it asynchronously via `await _db.SaveChangesAsync()` into the SQL Server database for regulatory auditing.

**6** **JSON Response & Dynamic Client Hydration (Angular Signals)**

The API returns HTTP 200 OK with the structured JSON `LoanRiskProfile`. In Angular, the RxJS subscription triggers: `this.result.set(data); this.loading.set(false); this.currentStep.set(3);`. Without reloading the page (AJAX), Angular's virtual DOM immediately transitions to Step 3, rendering visual risk badges, Sharpe ratio cards, VaR exposure charts, and bank likelihood explanations.

## 4 Web Technologies & Frontend Architecture

This project is built from the ground up to showcase state-of-the-art web technology standards. Each tier of the web stack was deliberately chosen and engineered according to modern industry specifications:

### 4.1 HTML (HyperText Markup Language — HTML5)

HTML forms the structural backbone of every page in CreditRiskSystem. Rather than relying on outdated table layouts or unsemantic `<div>` soup, HTML5 semantic standards are strictly followed:

**Semantic Structural Elements:** Extensive use of `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, and `<footer>` across all pages to guarantee accessibility, search engine readability, and clear DOM tree hierarchy.
**Angular 18 Modern Control Flow Syntax:** Templates utilize Angular's next-generation declarative control flow blocks:
`@if (loading()) { ... } @else { ... }` for conditional rendering of loading skeletons versus computed results.
`@for (model of filteredModels; track model.id) { ... }` for high-performance DOM node recycling when listing financial models.
`@switch (activeSection()) { @case ('analytics') { ... } }` for multi-tab administrator views.
**HTML5 Form Controls:** Optimized input types (`type="number"`, `min`, `max`, `step`, `type="password"`) enforcing browser-level data integrity and accessible user interaction before submission.
**Data Binding Directives:** Property binding (`[disabled]="!canProceed()"`), Event binding (`(click)="analyse()"`), and Two-way binding (`[(ngModel)]="form.annualIncome"`).

### 4.2 CSS & SCSS (Cascading Style Sheets / SASS)

The visual presentation is powered by custom SCSS implementing a **Glassmorphic Dark-Themed Design System**. No generic templates or bloated utility libraries are used:

**CSS Custom Properties (Variables):** Defined in `styles.scss` to maintain global consistency:

```
:root {
  --bg-primary: #080c14;
  --bg-card: rgba(15, 23, 42, 0.7);
  --accent-teal: #14b8a6;
  --accent-amber: #f59e0b;
  --accent-purple: #8b5cf6;
  --border-glass: rgba(255, 255, 255, 0.08);
}
```

**Glassmorphism & Depth:** Components utilize `backdrop-filter: blur(16px)` combined with translucent RGBA background gradients and subtle 1px border highlights to achieve a futuristic, multi-layered visual depth.
**Responsive Layout Systems:** Multi-column layouts are orchestrated via **CSS Flexbox** (`display: flex; justify-content: space-between;`) and **CSS Grid** (`grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));`), guaranteeing flawless rendering across desktops, laptops, tablets, and smartphones.
**Micro-Animations & Keyframes:** Smooth CSS transitions (`transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);`) and custom keyframe animations:

```
@keyframes pulseGlow {
  0%, 100% { box-shadow: 0 0 15px rgba(20, 184, 166, 0.2); }
  50% { box-shadow: 0 0 30px rgba(20, 184, 166, 0.5); }
}
```

**Component Style Encapsulation:** Angular SCSS files (`admin.scss`, `apply.scss`) use Emulated View Encapsulation, ensuring component styles never unintentionally bleed into global elements.

### 4.3 JavaScript (ECMAScript 6+ / ESNext Runtime)

JavaScript is the execution engine that powers all client-side logic in the browser:

**Browser Event Loop & Execution Context:** Manages user click events, input keystrokes, and asynchronous HTTP task scheduling across microtask and macrotask queues.
**Complex Mathematical Operations:** Performs immediate mathematical operations on numbers using built-in JS math functions:

```
// Example from apply.ts for monthly EMI calculation:
const r = (this.form.interestRate / 100) / 12;
const n = this.form.loanTermYears * 12;
const emi = (this.form.loanAmount * r * Math.pow(1 + r, n)) / (Math.pow(1 + r, n) - 1);
```

**Modern ES6+ Features:** Destructuring assignments, spread syntax ( `{ ...this.form }` ), template literals ( `` `${API}/sharpe` `` ), arrow functions, and array functional methods ( `.map()` , `.filter()` , `.reduce()` ).
**Production Compilation & Bundling:** Angular CLI compiles TypeScript code down to highly optimized, minified, and tree-shaken JavaScript bundles ( `chunk-WL2ISWI7.js` , `main.js` ) that load instantaneously in any modern web browser.

### 4.4 AJAX (Asynchronous JavaScript and XML) & Asynchronous Web Communication

**AJAX is one of the most critical technologies in this project.** In traditional pre-AJAX web applications, submitting a loan form required a full synchronous HTTP POST request, causing a complete browser screen refresh ( `F5` reload), disrupting user state, and re-downloading HTML, CSS, and images.

> **How AJAX is Implemented in CreditRiskSystem**
> In this project, AJAX is implemented using Angular's asynchronous `HttpClient` service, which wraps the browser's native `fetch` and `XMLHttpRequest` APIs inside reactive **RxJS Observables**.

**Zero Page Reloads (Single Page Application):** When the user clicks *"Analyse Profile"*, the browser does not reload. An asynchronous HTTP POST request is sent in the background:

```
// In finance.ts (AJAX Service Layer)
getLoanRiskProfile(data: any): Observable<any> {
  return this.http.post(`${API}/loan-risk-profile`, data);
}

// In apply.ts (Component AJAX Subscription)
this.financeService.getLoanRiskProfile(payload).subscribe({
  next: (data) => {
    this.result.set(data);   // Updates UI reactively
    this.loading.set(false);
    this.currentStep.set(3);
  },
  error: (err) => {
    this.loading.set(false);
    console.error("AJAX Error:", err);
  }
});
```

**Non-Blocking User Experience:** The user interface remains active, animations continue smoothly, and a loading spinner communicates progress while the backend runs quantitative calculations.
**Dynamic Asynchronous Authorization:** The Admin portal uses AJAX to send JWT Bearer tokens inside the HTTP `Authorization` header asynchronously to verify admin sessions without redirecting the user.

### 4.5 XML (Extensible Markup Language) & JSON Usage

To address academic requirements on how XML and data serialization are utilized in modern web systems, the project clearly delineates the roles of both XML and JSON:

**JSON as the Primary Data Interchange Format:** While AJAX originally stood for *Asynchronous JavaScript and XML*, modern web industry standards have replaced XML data payloads with **JSON (JavaScript Object Notation)**. JSON is natively parsed by JavaScript with zero performance overhead and matches TypeScript object shapes directly ( `application/json` ). Every API communication in this project uses JSON.
**Where XML IS Actively Used in This Project:**
**SVG Vector Graphics (XML Markup):** All high-definition icons throughout the platform (e.g., in `navbar` , `home.html` , and `admin.html` ) are written in pure XML using the SVG 1.1 specification:

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="none" stroke="currentColor">
  <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M9 12l2 2 4-4m6 2a9 9 0 11-18 0 9 9 0 0118 0z" />
</svg>
```

**MSBuild Project Files:** The backend project configuration ( `CreditRisk.API.csproj` ) is an XML document defining target frameworks ( `<TargetFramework>net8.0</TargetFramework>` ), NuGet package dependencies, and build properties.
**Foundational XMLHttpRequest API:** The browser's underlying engine for AJAX communication originates from the native XML DOM interface.

### 4.6 TypeScript (Type-Safe Full-Stack Frontend Development)

TypeScript provides static type-safety, object-oriented design patterns, and next-generation reactivity across the entire UI codebase:

**Angular 18+ Signals for Reactive State Management:** Instead of legacy Zone.js change detection, modern Angular Signals (`signal()`, `computed()`) are used:

```
// Signal declaration
isLoggedIn = signal<boolean>(false);
analytics = signal<any>(null);

// Reactive update
this.isLoggedIn.set(true);
this.currentStep.update(step ⇒ step + 1);
```

Signals enable fine-grained, micro-targeted DOM re-rendering, resulting in 60fps UI performance.
**Strict Typing & Interface Contracts:** Ensures that financial inputs (numbers, interest rates, employment types) cannot be mistakenly handled as strings, eliminating runtime NaN errors.
**Modern Dependency Injection:** Employs TypeScript's functional `inject()` pattern (`private http = inject(HttpClient);`) for clean, testable service injection.
**Custom TypeScript Directives:** The project includes `ScrollRevealDirective` (`appScrollReveal`) in `directives/scroll-reveal.ts`. It interfaces with the browser's native `IntersectionObserver` API to add smooth reveal animations as sections enter the user viewport.

## 5 Database & Backend Architecture

The server-side layer is built with **ASP.NET Core 8 Web API** and **Entity Framework Core 8**, connecting to **Microsoft SQL Server**.

#### 🗄️ Relational Database Schema (SQL Server)

- **Applicants Table:** Stores applicant credentials, email (unique index), annual income, and debt-to-income ratio with `decimal(18,2)` precision.
- **LoanApplications Table:** Tracks requested principal, interest rate, term, and lifecycle status (`Pending`, `Approved`, `Rejected`). Linked to Applicant via foreign key with restricted deletion.
- **RiskScores Table:** Stores historical and computed risk metrics with cascade delete rules linked to Loan Applications.
- **AnalysisLogs Table:** Regulatory compliance table storing complete input/output JSON audit blobs for every algorithmic decision.
- **LearnContents Table:** CMS table storing educational formulas, summaries, and examples for all 12 quantitative models.

#### 🔒 Security & Authentication Architecture

- **JWT Authentication:** Signed with HMAC-SHA256 tokens in `AdminController.cs`. Tokens include claims for `Admin` roles and automatic 8-hour expiry.
- **Role-Based Authorization:** Endpoints are protected with `[Authorize(Roles = "Admin")]`. Unauthorized calls return HTTP 401/403.
- **Session Countdown & Verification:** Angular frontend monitors remaining JWT validity and verifies authenticity with `/api/admin/verify` on startup.
- **CORS Policy:** Configured in `Program.cs` allowing secure cross-origin requests from the Angular client.

## 6 Institutional Quantitative Models Reference

The backend `FinanceEngine` implements 12 distinct mathematical finance calculators. Below is the reference sheet of models used in CreditRiskSystem:

| Model Name | Mathematical Formula | Financial Interpretation & Platform Role |
|---|---|---|
| **Sharpe Ratio** | `SR = (R_p - R_f) / σ_p` | Measures excess return earned per unit of total risk. Higher is superior. Used in loan profiling and comparing loan cost vs fund return. |
| **CAPM (Capital Asset Pricing Model)** | `E(R) = R_f + β * (R_m - R_f)` | Determines expected required return based on systematic risk (beta). Employment type sets beta (Salaried: 0.6, Business: 1.3). |
| **Kelly Criterion** | `f* = (p * b - q) / b` | Calculates the mathematically optimal fraction of bank capital to allocate to avoid risk of institutional ruin. |
| **Value at Risk (VaR)** | `VaR = Principal * σ * Z_α * √(t/365)` | Quantifies maximum monetary loss over a 30-day horizon at a 95% confidence level ( `Z = 1.645` ). |
| **Merton Structural Model** | `DD = [ln(V/D) + (μ - 0.5σ²)T] / (σ√T)` | Treats the borrower's equity as a call option on total assets. Computes Distance-to-Default (DD) from asset volatility. |
| **Black-Scholes Model** | `C = S*N(d1) - K*e^(-rT)*N(d2)` | Analytical closed-form pricing of financial options; used to price credit guarantees and collateral options. |
| **Geometric Brownian Motion (GBM)** | `dS_t = μS_t dt + σS_t dW_t` | Stochastic differential equation used in `/invest` to simulate 500 Monte Carlo NAV trajectories. |
| **SIP Future Value** | `FV = P * [((1 + i)^n - 1) / i] * (1 + i)` | Computes compounding accumulation of systematic monthly investments over any horizon. |
| **CAGR Formula** | `CAGR = (V_final / V_initial)^(1/t) - 1` | Calculates the smoothed annualized geometric growth rate of past financial assets. |
| **Composite Score** | `Score = 0.35(SR) + 0.35(DD) + 0.20(Kelly) + 0.10(CAPM)` | Unified 0-100 score weighing all signals to yield final **Approve / Review / Reject** decisions. |

## 7 Project Structure & Component Mapping

### Frontend Structure ( `credit-risk-ui/` )

```
credit-risk-ui/
├── src/
│   ├── app/
│   │   ├── components/navbar/     # Global Navigation
│   │   ├── directives/            # ScrollReveal directive
│   │   ├── pages/
│   │   │   ├── home/              # Hero, journeys, market card
│   │   │   ├── apply/             # Multi-step loan profiler
│   │   │   ├── invest/            # SIP, CAGR, NAV Monte Carlo
│   │   │   ├── compare/           # Loan vs Fund engine
│   │   │   ├── learn/             # Model knowledge base
│   │   │   └── admin/             # JWT auth, logs, market controls
│   │   ├── services/
│   │   │   └── finance.ts         # AJAX HttpClient REST service
│   │   ├── app.routes.ts          # Lazy-loaded route map
│   │   └── app.config.ts          # Angular client providers
│   ├── styles.scss                # Global glassmorphic design tokens
│   └── index.html                 # Single HTML5 host document
```

### Backend Structure ( `CreditRisk.API/` )

```
CreditRisk.API/
├── Controllers/
│   ├── AdminController.cs         # JWT auth, logs, analytics
│   ├── FinanceController.cs       # Risk profile & math endpoints
│   └── MarketController.cs        # Macroeconomic data feeds
├── Data/
│   └── AppDbContext.cs            # EF Core SQL Server DB context
├── FinanceEngine/
│   ├── FinanceEngineService.cs    # Orchestration service
│   └── Calculators/               # 12 Math calculation classes
├── Models/                        # Applicant, Loan, Risk, Logs
├── Services/
│   └── MarketDataServices.cs      # Live market data fetcher
├── appsettings.json               # DB connection & JWT keys
└── Program.cs                     # Middleware, DI, CORS & Auth
```

## 8 Viva Voce Defense & Technical Q&A

Below are essential viva examination questions covering the Web Technology aspects of the project, complete with high-scoring technical responses:

**Q1: How is AJAX used in your project, and what is the difference between a traditional web request and an AJAX request?**

**Answer:** In traditional web applications, submitting a form triggers a full HTTP POST that causes the entire browser window to reload (`F5` reload), discarding client state and re-rendering all assets. In our project, **AJAX (Asynchronous JavaScript and XML)** is implemented using Angular's `HttpClient` service. When a user clicks *"Analyse Profile"* on `/apply`, an asynchronous HTTP request is dispatched in the background without reloading the page. While the server processes the Merton, Sharpe, and VaR models, the UI displays a reactive loading spinner. When the JSON response arrives, Angular reactively updates the component's Signals, transitioning the DOM directly to the Results view seamlessly.

**Q2: Why do we say AJAX when you are transmitting JSON instead of XML? Is XML used anywhere in this project?**

**Answer:** Historically, when Jesse James Garrett coined the term AJAX in 2005, XML was the prevailing standard for data interchange. In modern web engineering, **JSON (JavaScript Object Notation)** has superseded XML for RESTful APIs because JSON is lightweight, faster to parse, and natively matches JavaScript objects. However, **XML is actively used in our project** in two distinct ways: (1) All scalable vector graphics (SVG icons) in our Angular templates are defined as valid XML documents with namespaces (`xmlns="http://www.w3.org/2000/svg"`), and (2) Our backend build configuration (`CreditRisk.API.csproj`) is an XML MSBuild file managing dependencies and build targets.

**Q3: What role does TypeScript play in the frontend, and how does it improve upon vanilla JavaScript?**

**Answer:** TypeScript is a strictly typed superset of JavaScript that compiles down to clean ES6 JavaScript. In a financial application where numerical precision is vital, vanilla JavaScript is prone to subtle bugs (e.g., string concatenation instead of addition: `"500000" + "9.5"`). TypeScript introduces compile-time type safety, custom interfaces (e.g., `SharpeInput`, `LoanRiskProfile`), and class decorators (`@Component`, `@Injectable`). Furthermore, we utilize **Angular 18 Signals** (`signal()`, `computed()`) in TypeScript to manage state reactively with zero Zone.js overhead.

**Q4: How did you implement responsive design and modern styling without third-party CSS frameworks like Bootstrap?**

**Answer:** We implemented a bespoke **SCSS Design System** utilizing CSS Custom Properties (CSS variables) for design tokens (colors, border radii, glassmorphism filters). Layout responsiveness is handled natively using **CSS Flexbox** for navigation bars and toolbars, and **CSS Grid** with `repeat(auto-fit, minmax(280px, 1fr))` for multi-card metric grids. Glassmorphism is achieved using `backdrop-filter: blur(16px)` with translucent RGBA border highlights. This avoids the heavy performance bloat of external CSS libraries while ensuring 100% custom visual control.

**Q5: How does the frontend communicate with the database, and how is security maintained?**

**Answer:** The frontend never communicates directly with the database. It communicates over HTTPS with the ASP.NET Core 8 Web API via RESTful JSON endpoints. The Web API uses **Entity Framework Core** (an Object-Relational Mapper) to map C# domain models (`Applicant`, `LoanApplication`, `AnalysisLog`) to SQL Server relational tables using parameterized queries that prevent SQL injection. For security, administrator routes are protected by **JWT (JSON Web Tokens)** signed with HMAC-SHA256, transmitted in HTTP `Authorization: Bearer <token>` headers.


Generated for Academic Submission • All Rights Reserved